

**Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий**

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида и ReLU по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - **Конфигурация А (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - **Конфигурация В (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и ассигасу.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
 - Пользователь вводит (A, B) из $[-10; 10]$.
 - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c_0; c_1]$, класс.
 - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант 4 ($c_0 = 7, c_1 = -7$)

A	B	$A \oplus B$
7	7	7
7	-7	-7
-7	7	-7
-7	-7	7

Ход работы

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

C0 = 7
C1 = -7
RANGE_MIN, RANGE_MAX = -10, 10

X_raw = np.array([[C0, C0], [C0, C1], [C1, C0],
[C1, C1]], dtype=float)
Y_raw = np.array([[C0], [C1], [C1], [C0]],
dtype=float)

def normalize(v):
    return (v - RANGE_MIN) / (RANGE_MAX -
RANGE_MIN)

def to_original(a):
    return C1 + a * (C0 - C1)

X = normalize(X_raw)
Y_bin = np.array([[1.0] if y[0] == C0 else [0.0]
for y in Y_raw])

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_deriv(a):
    return a * (1.0 - a)

def relu(x):
    return np.maximum(0, x)

def relu_deriv(x):
    return (x > 0).astype(float)

def bce_loss(a, y):
    eps = 1e-9
    a = np.clip(a, eps, 1 - eps)
    return -(y * np.log(a) + (1 - y) * np.log(1 -
a))

class MLP:
    def __init__(self,
hidden_activation="sigmoid", lr=0.5, seed=0):
        rng = np.random.default_rng(seed)
        self.hidden_activation =
hidden_activation
        self.lr = lr
        self.W1 = rng.uniform(-0.5, 0.5, (2, 2))
        self.b1 = rng.uniform(-0.5, 0.5, (1, 2))
        self.W2 = rng.uniform(-0.5, 0.5, (2, 1))
        self.b2 = rng.uniform(-0.5, 0.5, (1, 1))

    def forward(self, x):
        x = x.reshape(1, -1)
        self.x = x
        self.z1 = x @ self.W1 + self.b1
        if self.hidden_activation == "relu":
            self.a1 = relu(self.z1)
        else:
            self.a1 = sigmoid(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def backward(self, target):
        target = target.reshape(1, -1)
        delta_out = self.a2 - target
        if self.hidden_activation == "relu":
            delta_hid = (delta_out @ self.W2.T) *
relu_deriv(self.z1)
        else:
            delta_hid = (delta_out @ self.W2.T) *
sigmoid_deriv(self.a1)
        self.W2 -= self.lr * (self.a1.T @
delta_out)
        self.b2 -= self.lr * delta_out
        self.W1 -= self.lr * (self.x.T @
delta_hid)
        self.b1 -= self.lr * delta_hid

    def train(self, X, Y, Ee, max_epochs,
record_history=False):
        history = [] if record_history else None
        for epoch in range(1, max_epochs + 1):
            Es = 0.0
            order = np.random.permutation(len(X))
            for i in order:
                out = self.forward(X[i])
                Es += bce_loss(out, Y[i]).item()
                self.backward(Y[i])
            if record_history:
                history.append(Es)
            if Es <= Ee:
                return epoch, Es, history
        return max_epochs, Es, history

    def predict(self, x):
        return
self.forward(np.array(x)).flatten()[0]

def evaluate(model):
    correct = 0
    abs_errors = []
    for i in range(len(X)):
        a = model.predict(X[i])
        y_orig_pred = to_original(a)
        pred_class = C0 if a >= 0.5 else C1
        target_class = Y_raw[i][0]
        correct += pred_class == target_class
        abs_errors.append(abs(y_orig_pred -
target_class))
    accuracy = correct / len(X)
    mae = np.mean(abs_errors)
    return accuracy, mae

def run_network(model, A, B):
    x_norm = normalize(np.array([A, B],
dtype=float))
    a_norm = model.predict(x_norm)
    y_orig = to_original(a_norm)
    cls = C0 if a_norm >= 0.5 else C1
    return a_norm, y_orig, cls

Ee = 0.01
MAX_EPOCHS = 50000
LR = 0.5
```

```

SEEDS = [0, 1, 2, 3, 4]

results_A = []
results_B = []
history_A0 = None
history_B0 = None
model_A0 = None
model_B0 = None

for seed in SEEDS:
    record = seed == 0

    model_A = MLP(hidden_activation="sigmoid",
lr=LR, seed=seed)
    epochs_A, Es_A, hist_A = model_A.train(X,
Y_bin, Ee, MAX_EPOCHS, record)
    acc_A, mae_A = evaluate(model_A, A")
    results_A.append((epochs_A, Es_A, acc_A,
mae_A))
    if record:
        history_A0 = hist_A
        model_A0 = model_A

    model_B = MLP(hidden_activation="relu", lr=LR,
seed=seed)
    epochs_B, Es_B, hist_B = model_B.train(X,
Y_bin, Ee, MAX_EPOCHS, record)
    acc_B, mae_B = evaluate(model_B)
    results_B.append((epochs_B, Es_B, acc_B,
mae_B))
    if record:
        history_B0 = hist_B
        model_B0 = model_B

def print_results(name, results):
    print(f"\n{name}")
    for i, (ep, es, acc, mae) in
enumerate(results):
        print(f"seed={i} | эпох={ep:<7} |
Es={es:.5f} | acc={acc*100:.0f}% | MAE={mae:.3f}")
        epochs_arr = np.array([r[0] for r in results])
        es_arr = np.array([r[1] for r in results])
        acc_arr = np.array([r[2] for r in results])
        mae_arr = np.array([r[3] for r in results])
        print(f"среднее: эпох={epochs_arr.mean():.0f}
(std={epochs_arr.std():.0f}) | "
f"Es={es_arr.mean():.5f}
acc={acc_arr.mean()*100:.0f}%
MAE={mae_arr.mean():.3f}")

    print_results("Конфигурация A (Sigmoid)",
results_A)
    print_results("Конфигурация B (ReLU)", results_B)

    print("\nРЕЖИМ ФУНКЦИОНИРОВАНИЯ (конфигурация B,
ReLU)")
    demo_pairs = [(7, 7), (7, -7), (-7, 7), (-7, -7),
(8, -6), (0, 0), (3, -9)]
    for A, B in demo_pairs:
        a_norm, y_orig, cls = run_network(model_B0, A="black",
B)
        print(f"A={A:>4} B={B:>4} | выход
norm={a_norm:.3f} | выход [c0;c1]={y_orig:.3f} | класс
{cls}")

    print("\nВВОД ПОЛЬЗОВАТЕЛЯ ('q' для выхода)")
    while True:
        a_in = input("A: ").strip()
        if a_in.lower() == 'q':
            break
        b_in = input("B: ").strip()
        if b_in.lower() == 'q':
            break
        try:
            a_norm, y_orig, cls =
run_network(model_B0, float(a_in), float(b_in))
            print(f"выход norm={a_norm:.3f} | выход
[c0;c1]={y_orig:.3f} | класс {cls}\n")
        except ValueError:
            print("Некорректный ввод\n")

plt.figure(figsize=(7, 5))
plt.plot(history_A0, label="Sigmoid (конфигурация
A)")
plt.plot(history_B0, label="ReLU (конфигурация
B)")
plt.xlabel("Эпоха")
plt.ylabel("Суммарная ошибка Es (BCE)")
plt.title("Сходимость обучения")
plt.legend()
plt.tight_layout()
plt.savefig("convergence.png")
plt.close()

epochs_A_list = [r[0] for r in results_A]
epochs_B_list = [r[0] for r in results_B]
x_idx = np.arange(len(SEEDS))
width = 0.35

plt.figure(figsize=(7, 5))
plt.bar(x_idx - width / 2, epochs_A_list, width,
label="Sigmoid (A)")
plt.bar(x_idx + width / 2, epochs_B_list, width,
label="ReLU (B)")
plt.xticks(x_idx, [f"seed {s}" for s in SEEDS])
plt.ylabel("Эпох до сходимости")
plt.title("Число эпох по 5 запускам")
plt.legend()
plt.tight_layout()
plt.savefig("epochs_bar.png")
plt.close()

def plot_heatmap(model, filename, title):
    grid = np.linspace(RANGE_MIN, RANGE_MAX, 200)
    AA, BB = np.meshgrid(grid, grid)
    Z = np.zeros_like(AA)
    for i in range(AA.shape[0]):
        for j in range(AA.shape[1]):
            x_norm = normalize(np.array([AA[i, j],
BB[i, j]]), dtype=float))
            Z[i, j] = model.predict(x_norm)

plt.figure(figsize=(6, 5))
plt.contourf(AA, BB, Z, levels=50,
cmap="coolwarm")
plt.colorbar(label="Выход сети (норм.)")
plt.scatter(X_raw[:, 0], X_raw[:, 1],
marker="x", s=80, label="Обучающие точки")
plt.xlabel("A")
plt.ylabel("B")
plt.title(title)
plt.legend()
plt.tight_layout()
plt.savefig(filename)
plt.close()

```

```

        plot_heatmap(model_B0, "heatmap_B.png",
        "Разделяющая поверхность (ReLU, B)")
plot_heatmap(model_A0, "heatmap_A.png",
"Разделяющая поверхность (Sigmoid, A)")
print("\nГрафики сохранены: convergence.png,
epochs_bar.png, heatmap_A.png, heatmap_B.png")

```

Вывод программы:

```

Конфигурация A (Sigmoid)
seed=0 | эпох=2607 | Es=0.01000 | acc=100% | MAE=0.035
seed=1 | эпох=2690 | Es=0.01000 | acc=100% | MAE=0.035
seed=2 | эпох=2648 | Es=0.01000 | acc=100% | MAE=0.035
seed=3 | эпох=2891 | Es=0.01000 | acc=100% | MAE=0.035
seed=4 | эпох=2864 | Es=0.01000 | acc=100% | MAE=0.035
среднее: эпох=2740 (std=116) | Es=0.01000 | acc=100% | MAE=0.035

Конфигурация B (ReLU)
seed=0 | эпох=50000 | Es=1.51918 | acc=75% | MAE=3.500
seed=1 | эпох=50000 | Es=2.08905 | acc=75% | MAE=4.653
seed=2 | эпох=50000 | Es=2.07771 | acc=75% | MAE=4.689
seed=3 | эпох=50000 | Es=3.03846 | acc=50% | MAE=7.000
seed=4 | эпох=50000 | Es=2.07867 | acc=75% | MAE=4.609
среднее: эпох=50000 (std=0) | Es=2.16061 | acc=70% | MAE=4.890

РЕЖИМ ФУНКЦИОНИРОВАНИЯ (конфигурация B, ReLU)
A= 7 B= 7 | выход погт=0.501 | выход [c0;c1]=0.014 | класс 7
A= 7 B= -7 | выход погт=0.000 | выход [c0;c1]=-7.000 | класс -7
A= -7 B= 7 | выход погт=0.501 | выход [c0;c1]=0.014 | класс 7
A= -7 B= -7 | выход погт=1.000 | выход [c0;c1]=7.000 | класс 7
A= 8 B= -6 | выход погт=0.000 | выход [c0;c1]=-7.000 | класс -7
A= 0 B= 0 | выход погт=0.501 | выход [c0;c1]=0.014 | класс 7
A= 3 B= -9 | выход погт=0.002 | выход [c0;c1]=-6.970 | класс -7

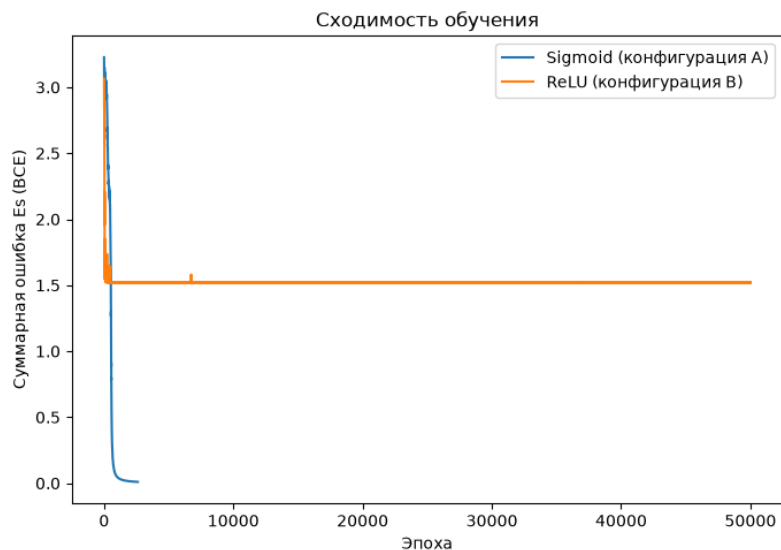
ВВОД ПОЛЬЗОВАТЕЛЯ ('q' для выхода)
A: 7
B: -7
выход погт=0.000 | выход [c0;c1]=-7.000 | класс -7

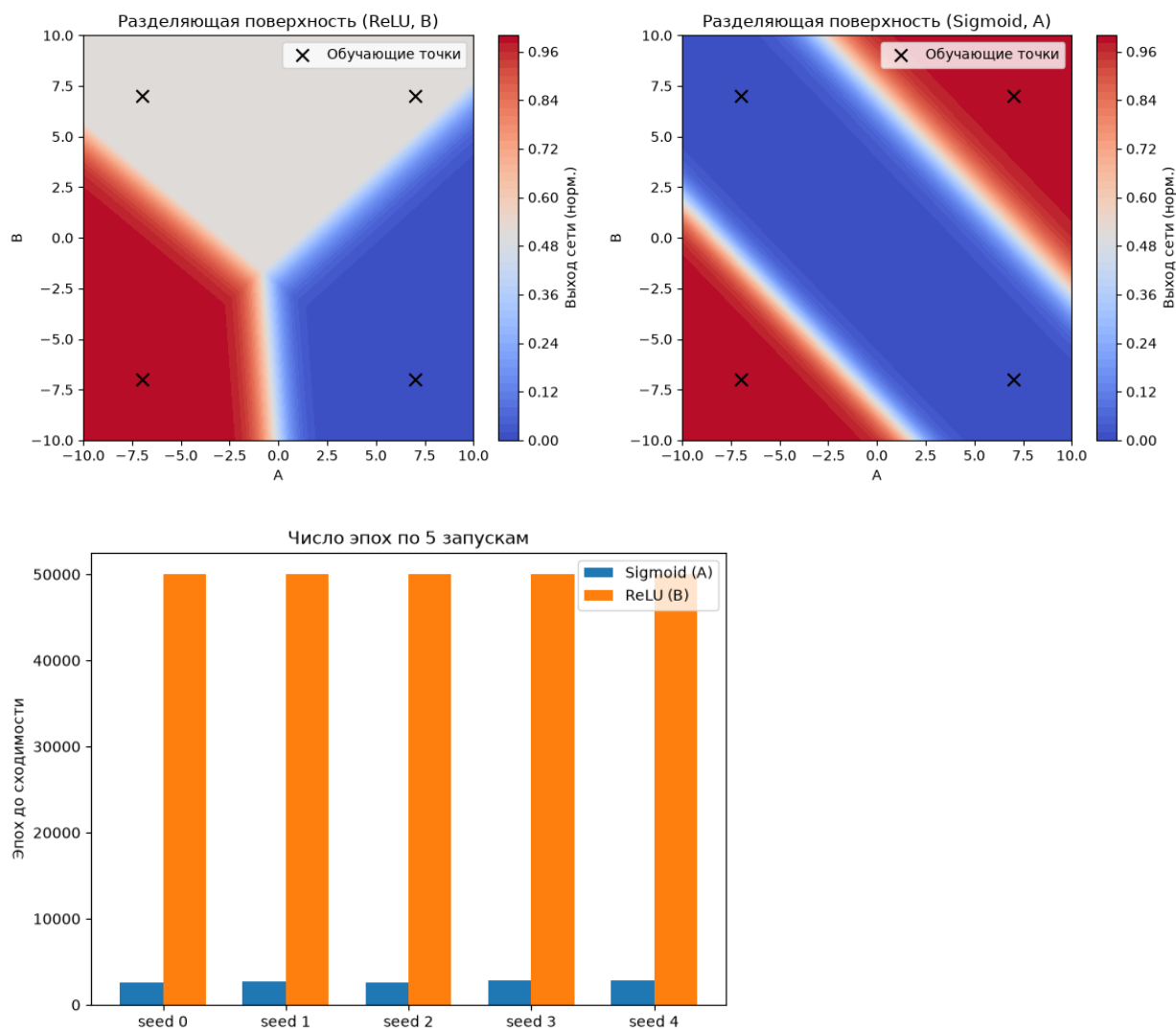
A: 0
B: 0
выход погт=0.501 | выход [c0;c1]=0.014 | класс 7

A: -10
B: -10
выход погт=1.000 | выход [c0;c1]=7.000 | класс 7

```

Графики:





Сигмоида показала более быструю и устойчивую сходимость: конфигурация А сошлась в среднем за 2740 эпох с точностью 100% и MAE=0.035 на всех пяти запусках, тогда как ReLU ни разу не достигла критерия останова за 50000 эпох, оставшись с ошибкой около 2.16 и точностью всего 70%. Причина видна в режиме функционирования: для входов (7, 7), (-7, 7) и (0, 0) сеть выдала одинаковый выход 0.501, хотя эти точки должны классифицироваться по-разному. Это признак "мёртвого" ReLU-нейрона, который перестал активироваться и выпал из обучения, из-за чего сеть фактически работала одним скрытым нейроном вместо двух и не смогла построить вторую границу, нужную для XOR. Поэтому способность решать XOR при переходе на ReLU снизилась, а разделяющая поверхность перестала совпадать с ожидаемой диагональной структурой классов. Это противоречит обычному преимуществу ReLU в глубоких сетях, где она ускоряет обучение за счёт отсутствия затухающего градиента, но в такой маленькой архитектуре с всего двумя скрытыми нейронами риск "смерти" нейрона оказался выше пользы, поэтому сигмоида оказалась надёжнее. В целом эксперимент подтверждает, что скрытый слой необходим для решения XOR независимо от активации, но выбор активации сильно влияет на то, реализуется ли этот потенциал на практике: с сигмоидой оба нейрона остаются рабочими, а с ReLU часть из них может выпасть из обучения, и тогда архитектура перестаёт давать ожидаемый результат.

Вывод: на основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовала влияние активационной функции ReLU на скрытых слоях. Провела сравнительный анализ обучения сети с функцией активации Сигмоида и ReLU по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.